

Design and Development of an Intelligent Chronic Kidney Disease (CKD) Risk Prediction and Clinical Patient Management System

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
In
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

JAYAVARAPU SRI CHARAN : 2520090137

JADALA SOHAN : 2520090137

ADITHYA : 2520080036

Under the Esteemed Guidance of

Dr. M.Gayatri

Assistant Professor

Department of CSIT and AI&DS



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF AI&DS
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled "Design and Development of an Intelligent Chronic Kidney Disease (CKD) Risk Prediction and Clinical Patient Management System" is a record of Bonafide work of JAYAVARAPU SRI CHARAN - 2520090137, JADALA SOHAN - 2520090137, and ADITHYA - 2520080036 submitted in partial fulfillment for the award of B. Tech in AI&DS and Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

JAYAVARAPU SRI CHARAN – 2520090137
JADALA SOHAN – 2520090137
ADITHYA – 2520080036
Department of CSIT and AI&DS

**K L (Deemed to be) University
DEPARTMENT OF CSIT**

&

DEPARTMENT OF AI&DS



CERTIFICATE

This is to certify that the mini project based report entitled "Design and Development of an Intelligent Chronic Kidney Disease (CKD) Risk Prediction and Clinical Patient Management System" is a bonafide work done and submitted by JAYAVARAPU SRI CHARAN - 2520090137, JADALA SOHAN - 2520090137, and ADITHYA - 2520080036 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of CSIT and AI&DS Engineering, K L (Deemed to be University), during the academic year 2025-2026.

**Signature of the Guide
Dr. M.Gayatri**

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator Dr. M.Gayatri and our guide Dr.M.Gayatri, Department of CSIT and Engineering, which cannot be expressed in words for his tremendous support, encouragement, and invaluable guidance throughout the conception and implementation of this project.

We express our profound gratitude to Dr.M.Gayatri/ Dr. Kaja Shareef (CSIT), Head of the Department for Computer Science Engineering / Department for Computer Science and Information Technology, for providing us with state-of-the-art computational infrastructure, laboratories, and resources by which we were able to complete this project-based Lab curriculum.

We thank all the members of teaching and non-teaching staff members who have assisted directly or indirectly in validating our database designs and cloud architectures. Finally, we sincerely thank our parents, family, and classmates for their constant motivation, kind help, and cooperation during our engineering studies.

JAYAVARAPU SRI CHARAN – 2520090137

JADALA SOHAN – 2520090137

ADITHYA – 2520080036

Department of CSIT and AI&DS

K L University

Abstract

Chronic Kidney Disease (CKD) represents one of the most critical and fast-growing health burdens worldwide, affecting over 850 million people and contributing significantly to global morbidity and mortality. The disease is characterized by a gradual loss of kidney function over time, frequently manifesting with minimal or non-specific symptoms until advanced stages (Stages 4 and 5 / End-Stage Renal Disease). Traditional diagnostic workflows in hospitals remain fragmented: patient assessments, renal laboratory panels (serum creatinine, blood urea, urine protein, specific gravity), appointment bookings, and clinical evaluations are typically managed across disconnected legacy systems. This disconnect leads to diagnostic delays, transcription errors, clinical blind spots, and lost opportunities for timely medical interventions.

To overcome these multifaceted challenges, this project presents the “Design and Development of an Intelligent Chronic Kidney Disease (CKD) Risk Prediction and Clinical Patient Management System”. The platform merges advanced Machine Learning (ML) and Explainable Artificial Intelligence (XAI) with a rigorous Relational Database Management System (DBMS) and a full-stack, responsive web application. The core objective is to deliver a reliable, unified digital healthcare ecosystem that automates clinical triage, renal disease risk assessment, laboratory workflow execution, and multi-role hospital management under stringent medical safety standards.

The platform is anchored by four interconnected, role-isolated portals operating under robust Role-Based Access Control (RBAC): Patient Portal, Doctor Portal, Staff/Laboratory Portal, and Hospital Administrator Portal. Patients can submit structured 24-feature clinical intake assessments, schedule non-conflicting consultations, and review verified laboratory results alongside risk evaluations. Laboratory technicians manage test requisitions, input quantitative renal biomarkers (BUN, creatinine, electrolytes), upload diagnostic report files, and trigger atomic verification procedures. Nephrologists and clinicians benefit from a unified Patient 360 View, execute one-click machine learning inference with transparent SHAP (SHapley Additive exPlanations) risk-contribution charts, author digital prescriptions, and track longitudinal patient trajectories. Hospital administrators oversee system health, manage users, trace immutable audit logs, and execute live analytical SQL queries via a built-in query runner.

From a machine learning engineering standpoint, the predictive engine was developed and validated exclusively on the official UCI Chronic Kidney Disease benchmark dataset (400 instances across 24 clinical features including 14 numerical biomarkers and 10 categorical clinical observations). Under a rigorous 80/20 stratified train-test split with zero data leakage, both XGBoost and LightGBM Classifiers achieved 100.0% accuracy, precision, recall (sensitivity), F1-score, and ROC-AUC on unseen test instances. To ensure explainability in high-stakes clinical decision-making, the system embeds the SHAP TreeExplainer, quantifying the exact directional contribution of key renal indicators—such as Hemoglobin (hemo), Serum Creatinine (sc), Albuminuria (al), and Urine Specific Gravity (sg)—for each prediction.

The foundational data architecture is powered by a relational PostgreSQL database normalized to Third Normal Form (3NF), comprising 14 structured entities, Primary and Foreign Keys with cascading actions,

physiological CHECK constraints, composite UNIQUE constraints preventing doctor double-booking, B-tree performance indexes, analytical PostgreSQL views (v_patient_360, v_doctor_workload, v_lab_queue, v_prediction_analytics), ACID-compliant stored procedures (sp_book_appointment, sp_verify_lab_test), and automated audit triggers. The backend is built using high-performance asynchronous FastAPI (Python) and Uvicorn ASGI, secured with JWT authentication and bcrypt password hashing. In summary, this project delivers a secure, accessible, and mathematically sound digital health solution that enhances clinical workflow efficiency and sets a modern benchmark for trustworthy medical AI applications.

Index

S. No.	Chapters	Topics	Page.no
	Preliminary	Acknowledgement	iv
	Preliminary	Abstract	v
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	1 - 11
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	12 - 17
3	Technology Stack	3.1 Front-end (HTML5, CSS3, Modern JavaScript ES6+, Chart.js) 3.2 Back-end (FastAPI, Python, Uvicorn, Pydantic, Scikit-learn, XGBoost) 3.3 Database (PostgreSQL, 3NF Normalization, Views, Procedures, Triggers) 3.4 Version Control (Git, GitHub, Branching Strategy) 3.5 APIs or External services (RESTful APIs, JWT Auth, Swagger/OpenAPI, Docker)	18 - 28
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (Presentation, API Gateway, Application, ML, DBMS) 4.3 Deployment architecture (Containerization, Cloud hosting, CI/CD)	29 - 33
5	Design	5.1 System Data Flow & Architecture 5.2 ER Diagram / Relational Database Schema 5.3 Database Normalization Analysis (1NF, 2NF, 3NF)	34 - 39
6	Implementation	6.1 Module-wise implementation (Auth, Patient, Doctor, Lab, Admin, ML, SHAP) 6.2 Front-end logic (UI rendering, state management, tab routing) 6.3 API integration (REST endpoints, JSON contracts, Fetch requests) 6.4 Authentication & Authorization (JWT token lifecycle, RBAC, Bcrypt)	40 - 50
7	Features	7.1 List of main features 7.2 How each feature works (User perspective + Technical description)	51 - 59

8	Testing	8.1 Postman testing (Frontend/Backend API validation) 8.2 Integration testing (Multi-role clinical workflow verification) 8.3 Tools used for testing (PyTest, Postman, Newman, Selenium, JMeter) 8.4 Test cases and results (Structured test execution matrix)	60 - 69
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration (.env variables, secrets, database URLs) 9.3 Hosting of frontend and backend (Render, Docker, Uvicorn ASGI)	70 - 78
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	79 - 87
11	Future Enhancements	11.1 Planned features (FHIR/HL7 EHR, Mobile app, IoT sensor telemetry) 11.2 Possible integrations or optimizations (Federated learning, Transformers)	88 - 93
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	94 - 101
13	References	Textbooks, scientific journals, benchmark datasets, APIs, official documentation	102 - 104
14	Appendices	Appendix A: Screenshots of the application Appendix B: Sample code snippets (ML, SQL, FastAPI, Frontend) Appendix C: Installation and setup instructions Appendix D: End-to-end user manual and test guide	105 - 110

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Healthcare systems worldwide are experiencing unprecedented strain driven by the rising prevalence of chronic non-communicable diseases. Among these, Chronic Kidney Disease (CKD) represents an urgent global epidemic, affecting approximately 10% to 14% of the global adult population—exceeding 850 million individuals. Chronic Kidney Disease involves progressive and irreversible loss of renal nephron architecture and physiological filtration capacity over months or years. In its nascent phases (Stages 1 through 3), CKD is notoriously asymptomatic, earning it the clinical moniker of a 'silent killer'. Without early detection and clinical risk mitigation, CKD inevitably progresses to End-Stage Renal Disease (ESRD), requiring life-sustaining renal replacement therapy such as maintenance hemodialysis or organ transplantation.

The clinical and economic burden of advanced CKD is immense. Hemodialysis and renal transplants require specialized nephrology infrastructure, frequent hospitalizations, and catastrophic out-of-pocket healthcare expenditures. Conversely, extensive nephrological studies published by Kidney Disease: Improving Global Outcomes (KDIGO) confirm that if CKD is diagnosed during its initial asymptomatic stages, simple and cost-effective clinical interventions—including blood pressure normalization, angiotensin-converting enzyme inhibitors (ACEi/ARBs), glycemic control, dietary protein moderation, and cardiovascular risk reduction—can dramatically halt or reverse renal deterioration.

Despite the transformative value of early diagnosis, contemporary clinical workflows in outpatient clinics, diagnostic laboratories, and general hospital settings remain profoundly fragmented. Clinical data is compartmentalized into discrete silos: patient clinical history is documented in paper files or legacy electronic medical records (EMRs); quantitative renal biomarkers (serum creatinine, blood urea nitrogen, urine specific gravity, albuminuria, serum electrolytes) are processed in isolated clinical laboratory information systems (LIMS); and nephrology consultation scheduling is handled through manual desk registers or disconnected administrative portals. Consequently, primary care physicians and nephrologists lack a unified, real-time clinical window that correlates patient symptoms with laboratory biomarker trends and predictive analytics.

The idea for this project emerged from observing these critical healthcare inefficiencies and recognizing the opportunity to synthesize advanced Machine Learning (ML), Explainable Artificial Intelligence (XAI), and Relational Database Management Systems (DBMS) into a cohesive digital clinical platform. The proposed platform, entitled “Smart Chronic Kidney Disease (CKD) Risk Prediction and Patient Management System”, establishes a single-window digital healthcare environment that bridges clinical diagnostic prediction with end-to-end hospital operational workflows. By integrating automated clinical intake, multi-role access control, laboratory test verification, predictive machine learning inference, and transparent feature explainability, the platform transforms how renal care is delivered.

Technological breakthroughs in modern asynchronous web frameworks (such as FastAPI and ASGI Uvicorn), robust relational databases (PostgreSQL), and state-of-the-art gradient boosted decision tree algorithms (XGBoost and LightGBM) make full clinical integration feasible, secure, and performant. By

combining 3NF-normalized relational schemas with mathematical explainability engines (SHAP TreeExplainer), the system ensures both operational data integrity and clinical transparency—enabling medical professionals to comprehend precisely why an individual patient has been stratified into a specific renal risk category.

1.2 Problem Statement

In the modern digital healthcare era, hospitals and clinical laboratories have adopted diverse computer applications; however, the overall process of renal risk assessment, diagnosis, and patient management remains fragmented, inefficient, and error-prone. The existing digital healthcare ecosystem suffers from five fundamental problems:

- **1. Fragmented Data Silos & Disconnected Clinical Workflows:** Hospital departments operate in administrative isolation. Laboratories generate biochemical reports independently, doctors maintain isolated outpatient records, and patients have no centralized mechanism to view their longitudinal health data or schedule timely follow-up appointments.
- **2. Diagnostic Latency in Asymptomatic Renal Decline:** Early-stage CKD is characterized by subtle physiological shifts across multiple interrelated biomarkers (e.g. slight drops in hemoglobin and urine specific gravity accompanied by marginal elevations in serum creatinine and blood urea). Human clinicians, overburdened by heavy patient volumes, frequently overlook these subtle multivariate correlations until overt renal impairment has occurred.
- **3. The 'Black-Box' Trust Deficit in Clinical AI:** Many modern machine learning algorithms implemented in academic research provide binary diagnostic classifications without clinical justification. In high-stakes medicine, clinicians cannot ethically or legally act upon opaque predictions. Without transparent biomarker feature attribution, algorithmic predictions cannot be integrated into clinical decision support.
- **4. Data Integrity Vulnerabilities & Double-Booking Failures:** Legacy outpatient management systems frequently fail to enforce strict ACID transactional consistency, leading to appointment scheduling collisions (double-booking the same physician at identical time slots), orphan test requisitions, and unvalidated biomarker entries outside physiological limits.
- **5. Inadequate Role-Based Access Control and Security Vulnerabilities:** Healthcare data is subject to strict privacy mandates (such as HIPAA and regional health data protection laws). Uncontrolled multi-user interfaces expose sensitive patient diagnostics to unauthorized administrative or auxiliary staff without proper cryptographic authentication or immutable audit trails.

In summary, the problem statement identifies a critical void: the absence of a unified, explainable, and transactional software platform that seamlessly connects the patient intake experience, laboratory verification pipeline, explainable machine learning risk assessment, and clinical prescription lifecycle under a secure, audited architecture.

1.3 Scope of the Project

The scope of the “Smart Chronic Kidney Disease (CKD) Risk Prediction and Patient Management System” extends across clinical, technological, operational, and data governance dimensions. The project delivers an enterprise-grade full-stack web platform built from first principles:

- **Clinical Diagnostic Scope:** Incorporates all 24 standardized clinical attributes defined in the official UCI Chronic Kidney Disease benchmark dataset, comprising 14 quantitative physiological biomarkers (age, blood pressure, specific gravity, albumin, sugar, blood glucose random, blood urea, serum creatinine, sodium, potassium, hemoglobin, packed cell volume, white blood cell count, red blood cell count) and 10 categorical symptoms (red blood cells normal/abnormal, pus cells, pus cell clumps, bacteria, hypertension, diabetes mellitus, coronary artery disease, appetite, pedal edema, anemia).
- **Machine Learning & Explainable AI Scope:** Implements rigorous data preprocessing with zero data leakage (median imputation for numerical variables, mode imputation for categorical attributes, standard scaling, and one-hot encoding). Trains and evaluates both XGBoost Classifier and LightGBM Classifier under 80/20 stratified validation. Integrates SHAP TreeExplainer to produce patient-specific local feature importance bar charts.
- **Relational DBMS Scope:** Develops a comprehensive 3NF-normalized PostgreSQL relational database featuring 14 distinct entities, explicit Primary and Foreign Keys with cascading actions, physiological CHECK constraints, composite UNIQUE appointment constraints, B-tree indexes, analytical database views (v_patient_360, v_doctor_workload, v_lab_queue), ACID stored procedures, and audit triggers.
- **Multi-Role User Portals Scope:** Designs four distinct, responsive web interfaces: (1) Patient Portal for intake assessments, appointment requests, and diagnostic reviews; (2) Doctor Portal for consultation management, Patient 360 review, one-click ML inference, and prescription authoring; (3) Staff/Laboratory Portal for requisition queuing, biomarker value entry, diagnostic file uploads, and verification; and (4) Admin Portal for role governance, audit log tracing, and live SQL query execution.
- **Security & Governance Scope:** Implements JSON Web Token (JWT) stateless authentication with HS256 cryptographic signatures, bcrypt password hashing, role-based dependency injection guards, and zero-cache HTTP security headers.

1.4 Objectives of the Project

The primary objective of this project is to conceptualize, design, implement, and rigorously validate an intelligent, unified clinical management and predictive web ecosystem for Chronic Kidney Disease. The specific academic and technical objectives are as follows:

- **1. To design and normalize a 14-entity relational database schema adhering strictly to Third Normal Form (3NF) principles, guaranteeing transactional consistency, referential integrity, and elimination of data anomalies.**

- **2.** To develop and benchmark state-of-the-art machine learning models (XGBoost and LightGBM) on the official UCI CKD dataset, ensuring robust generalization, zero data leakage, and high empirical diagnostic accuracy.
- **3.** To implement SHAP (SHapley Additive exPlanations) TreeExplainer integration, enabling clinicians to visualize the exact mathematical contribution and directional risk impact of each biological biomarker.
- **4.** To construct a high-performance asynchronous RESTful API backend using FastAPI, Python 3.10+, and SQLAlchemy ORM, fully documented via interactive Swagger UI and ReDoc.
- **5.** To engineer four dedicated, responsive, and aesthetically modern web portals (Patient, Doctor, Lab Staff, Admin) using semantic HTML5, CSS3, ES6+ JavaScript, and Chart.js data visualization.
- **6.** To implement robust Role-Based Access Control (RBAC) with cryptographic JWT token authentication and bcrypt password salting, ensuring strict isolation between user permissions.
- **7.** To prevent appointment scheduling collisions through database-enforced composite unique constraints (doctor_id, preferred_date, preferred_time) and atomic stored procedures.
- **8.** To establish full clinical provenance and regulatory auditability through automated database triggers that log every user creation, lab modification, and prediction execution.
- **9.** To implement a live interactive SQL Query Runner within the administrative console to facilitate academic demonstration and grading of complex SQL joins, aggregations, and subqueries.

1.5 Importance of the Application

The medical, technological, and societal importance of the Smart CKD Risk Prediction & Patient Management System is multifaceted:

From a clinical healthcare perspective, the platform functions as an intelligent decision-support multiplier. By synthesizing complex multi-organ biochemical indicators into an intelligible risk score with SHAP-explained biomarker drivers, the application assists nephrologists and general practitioners in detecting silent kidney failure months before conventional clinical presentation. Early initiation of nephroprotective therapy directly prevents the onset of irreversible End-Stage Renal Disease, sparing patients from debilitating dialysis regimens and drastically reducing mortality.

From a database management standpoint, the project exemplifies how advanced relational database engineering underpins enterprise clinical systems. While simple applications rely on basic CRUD operations, this system demonstrates the practical execution of 3NF relational normalization, B-tree query indexing, multi-table analytical views, atomic stored procedures, and trigger-based audit logging. It safeguards sensitive medical data against corruption, race conditions, and unauthorized modifications.

From an ethical and regulatory perspective, the integration of Explainable AI (XAI) addresses the critical ethical mandate that medical artificial intelligence must not operate as an uninterpretable black box. By clearly articulating which specific lab values (e.g. elevated serum creatinine of 3.4 mg/dL combined with reduced hemoglobin of 9.4 g/dL) drove the prediction, the system fosters clinical trust, empowers patient education, and complies with modern clinical safety directives.

Societally and economically, the platform democratizes access to specialized nephrological assessment. By providing an automated intake and triage system, public healthcare centers and rural clinics can screen vulnerable populations—such as patients with long-standing diabetes and hypertension—and seamlessly route high-risk individuals to tertiary nephrology centers.

1.6 Target Users / Audience

The target users of the Smart CKD Risk Prediction & Patient Management System span four well-defined roles across the healthcare delivery ecosystem:

- **1. Registered Patients:** Individuals undergoing health checkups, diabetic/hypertensive patients requiring regular renal monitoring, and individuals presenting with early symptoms such as pedal edema or fatigue. Patients utilize the platform to submit structured health assessments, book consultations with nephrologists, and view their verified diagnostic lab panels with educational risk guidance.
- **2. Nephrologists and Medical Clinicians:** Licensed medical practitioners who require a comprehensive 'Patient 360 View' consolidating history, vitals, and laboratory panels. Clinicians invoke machine learning inference, analyze SHAP biomarker contributions, formulate diagnoses, author digital prescriptions, and schedule follow-ups.
- **3. Laboratory Technologists and Staff:** Medical laboratory professionals responsible for processing blood and urine specimens. Staff members access a live requisition queue, enter quantitative biochemical parameters (RFP panels), attach laboratory files, and execute atomic verification routines.
- **4. Hospital Administrators and Data Officers:** Healthcare managers responsible for system governance, user onboarding, role management, system health monitoring, audit trail inspection, and executing analytical queries on hospital operational metrics.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements specify the physical computing infrastructure essential to design, develop, train, test, and host the integrated CKD prediction and patient management platform. Because the architecture combines relational database operations with machine learning model training and asynchronous API request handling, hardware configurations are categorized into developer workstations, production server environments, and end-user client devices.

- **Developer Workstation:** Multi-core 64-bit processor (Intel Core i5/i7 10th Gen or AMD Ryzen 5/7 with 6+ cores and clock speed ≥ 2.5 GHz); Minimum 8 GB RAM (16 GB recommended for concurrent Docker execution, local PostgreSQL server, FastAPI ASGI worker, and SHAP TreeExplainer computations); Storage of at least 256 GB NVMe SSD for fast compilation, virtual environment package management, and dataset I/O; Full HD display (1920x1080) for multi-portal frontend layout development; Broadband internet connectivity (≥ 20 Mbps) for package installations and Git synchronization.
- **Production Server Infrastructure:** Cloud virtual machine instance (such as AWS EC2 t3.medium or Render Standard Web Service) with dual-core virtual CPU (vCPU), 4 GB RAM, and 40 GB SSD persistent storage; Dedicated managed PostgreSQL database instance (such as AWS RDS or Render Managed PostgreSQL) with 1 GB RAM and automatic snapshot storage; Network bandwidth supporting HTTPS TLS termination with low-latency response times (< 150 ms).
- **End-User Client Devices:** Standard desktop, laptop, tablet, or smartphone equipped with a dual-core processor, 2 GB RAM, and any standard modern web browser. Client machines require minimal local storage (< 50 MB for cached browser assets) since all computations and database queries execute on the backend server.

2.2 Software Requirements

Software requirements outline the operating systems, runtime environments, libraries, database engines, and networking software necessary for the reliable execution of the platform across development and deployment:

- **Operating System:** Compatible across Windows 10/11, Ubuntu Linux 22.04 LTS, and macOS Monterey/Ventura/Sonoma. Ubuntu Linux 22.04 LTS is the selected production hosting operating system.
- **Runtime Environment:** Python 3.10, 3.11, or 3.12 64-bit runtime environment. Python provides native support for scientific computing, machine learning libraries, asynchronous web servers, and database connectors.
- **Web Framework:** FastAPI (version $\geq 0.110.0$), powered by Starlette and Pydantic v2. ASGI server execution is managed by Uvicorn (version $\geq 0.28.0$) supporting asynchronous non-blocking event loops.

- Database Management System: PostgreSQL 14.x / 15.x / 16.x Relational Database Management System supporting ANSI SQL, 3NF schemas, foreign key cascades, triggers, stored procedures, and JSONB document storage. (SQLite 3.x is supported as a zero-configuration local developer fallback).
- Machine Learning Libraries: Scikit-learn ($\geq 1.4.0$) for preprocessing, pipelines, and evaluation metrics; XGBoost ($\geq 2.0.0$) for primary gradient boosted decision tree classification; LightGBM ($\geq 4.3.0$) for secondary ensemble benchmarking; SHAP ($\geq 0.45.0$) for TreeExplainer local interpretability; Joblib ($\geq 1.3.0$) for model serialization.
- Security & Cryptography: Python-Jose ($\geq 3.3.0$) for JWT token encoding and validation; Passlib with Bcrypt ($\geq 4.1.0$) for cryptographically secure password hashing; Cryptography ($\geq 42.0.0$) for SSL/TLS handshakes.
- Client-Side Technologies: Semantic HTML5, CSS3 with Flexbox and CSS Grid, ECMAScript 6+ (Vanilla JS), Chart.js (v4.x) for dynamic canvas-based charting, and FontAwesome / Bootstrap Icons for clinical icons.

2.3 Development Tools and Frameworks

A modern toolchain was employed to ensure software quality, rapid debugging, and reliable version control throughout the project lifecycle:

- Integrated Development Environment (IDE): Visual Studio Code (VS Code) with extensions for Python, Pylance, SQL Tools, SQLite Viewer, GitLens, and HTML/CSS Formatting.
- API Testing & Documentation: Postman (v10.x) for automated endpoint request testing, variable injection, and assertion suites; Swagger UI and ReDoc (built into FastAPI) for interactive browser-based OpenAPI specification testing.
- Database Administration: PgAdmin 4 and DBeaver Community Edition for database diagram generation, table inspection, stored procedure debugging, and query optimization analysis.
- Version Control & Collaboration: Git distributed version control system hosted on GitHub ('2520090137_DBS'), employing semantic commit messages ('feat:', 'fix:', 'refactor:', 'docs:') and branch isolation.
- Containerization: Docker and Docker Compose for encapsulating application runtimes, database instances, and network bindings into reproducible, environment-agnostic containers.

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end tier of the Smart CKD Healthcare Platform was designed to deliver a fast, responsive, role-differentiated, and accessible clinical user experience across desktop monitors, medical workstations, tablets, and smartphones. Rather than adopting heavy, opinionated single-page application frameworks with large bundle sizes, the platform leverages lightweight semantic HTML5, modern modular CSS3, and native ECMAScript 6+ (Vanilla JavaScript).

This design choice was deliberate: in clinical healthcare environments, web interfaces must load instantaneously without compilation lags or client-side hydrate delays. Semantic HTML5 markup ensures accessibility and screen-reader compatibility across medical intake forms. Custom CSS3 styles, structured around a clinical color palette (Medical Navy `#1A365D`, Clinical Teal `#319795`, Alert Coral `#E53E3E`, and Soft Slate `#F7FAFC`), implement CSS Flexbox and CSS Grid layouts that adapt fluidly to screen dimensions from 360 pixels to 4K displays.

Client-side interactivity is handled via asynchronous JavaScript (Fetch API with `async/await` syntax), managing DOM manipulation dynamically without triggering disruptive full-page reloads. Modular JavaScript files (`api.js`, `config.js`) encapsulate API communications, attach Bearer authentication tokens, and route responses. For clinical data visualization, Chart.js is integrated to render dynamic radar charts of patient biomarker distributions, historical risk trajectories, and horizontal bar charts of SHAP risk contributions.

3.2 Back-End Design

The back-end service layer is engineered using FastAPI, a modern, high-performance Python web framework based on Starlette and Pydantic. FastAPI was selected because of its native support for asynchronous Python (`async def`), high throughput matching Node.js and Go, and automatic OpenAPI schema generation.

The backend architecture strictly follows the Model-View-Controller / Service-Repository software design pattern:

- **Controllers / Routers:** Organized into functional domain modules under `backend/app/routes/` (`auth.py`, `patients.py`, `doctors.py`, `staff.py`, `appointments.py`, `laboratory.py`, `predictions.py`, `admin.py`, `notifications.py`). Routers define RESTful endpoints, enforce role dependencies, and return validated Pydantic models.
- **Validation Schemas:** Defined under `backend/app/schemas/schemas.py`, utilizing Pydantic v2 to validate all incoming HTTP payloads against rigorous typing, regex constraints, and clinical value boundaries before reaching the database.
- **Service & ORM Layer:** Powered by SQLAlchemy ORM with declarative mappings in `backend/app/models/entities.py`, providing object-relational mapping, connection pooling, and transaction lifecycle management.

- Machine Learning Inference Engine: Encapsulated within `ml/explainability/explainer.py` and `backend/app/services/ml_service.py`, loading trained models (`best_model.joblib`) into memory on startup and executing sub-millisecond inference paired with SHAP TreeExplainer computations.

3.3 Database Design

The database management tier serves as the bedrock of the entire application, designed to ensure the highest standards of data integrity, relational consistency, and query efficiency. The primary database is PostgreSQL, an enterprise-grade, ACID-compliant relational database management system.

The schema is strictly normalized up to Third Normal Form (3NF), separating concerns across 14 relational tables (Roles, Users, Patients, Doctors, Staff, Doctor-Patient Assignments, Appointments, Medical Assessments, Laboratory Tests, Laboratory Results, Prediction Records, Clinical Notes, Notifications, and Audit Logs). Referential integrity is enforced through Foreign Key relationships with explicit `ON DELETE CASCADE` and `ON DELETE SET NULL` constraints.

To optimize data access and support advanced academic DBMS evaluation, the database integrates:

- B-Tree Indexes: Configured across high-frequency search fields, including user email addresses, patient identifiers, appointment schedules, and test statuses (`database/indexes.sql`).
- Materialized & Virtual Views: Providing pre-joined, real-time summaries such as `v_patient_360` (comprehensive clinical overview), `v_doctor_workload` (appointment and review statistics), and `v_lab_queue` (pending test requisitions).
- Stored Procedures & Functions: Encapsulating atomic business logic, such as `sp_book_appointment` (which verifies slot availability and prevents double-booking within an ACID transaction) and `sp_verify_lab_test` (which updates test statuses and cascades notifications).
- Audit Triggers: Automated database triggers (`trg_audit_user_registration`, `trg_audit_prediction`) that record immutable audit entries upon user creation and prediction generation.

3.4 Version Control

Version control was maintained using Git, with repository hosting and collaboration managed on GitHub under the repository `https://github.com/sricharan5865/2520090137_DBS`. The development lifecycle adhered to the GitFlow branching paradigm:

- The `main` branch was preserved as the production-ready release branch, containing only tested, verified, and stable builds.
- Feature development was isolated in dedicated branches (`feature/ml-pipeline`, `feature/rbac-portals`, `feature/dbms-views-procedures`), ensuring that unstable experimental code never compromised the core system.
- Standardized semantic commit conventions (`feat:`, `fix:`, `refactor:`, `docs:`, `test:`) were enforced to maintain a clean, traceable historical audit log.
- A comprehensive `.gitignore` file prevented sensitive credentials, `.env` files, virtual environments, bytecode caches, and temporary SQLite databases from being checked into public version control.

3.5 APIs or External Services

The application architecture exposes a comprehensive suite of RESTful API endpoints adhering to OpenAPI 3.0 standards. Key API and service integrations include:

- Authentication & Token Service: Exposes `/api/auth/login`` and `/api/auth/register``, generating signed JSON Web Tokens (JWT) with user ID, username, and role claims.
- Machine Learning Inference API: Exposes `/api/predictions/predict``, accepting patient assessment data, running the preprocessor pipeline, querying the XGBoost classifier, and returning prediction probabilities with top SHAP feature impact drivers.
- DBMS Live Query Runner API: Exposes `/api/admin/db/query``, an administrative endpoint allowing authorized evaluators to execute SQL statements directly against the database with strict injection filtering.
- Swagger UI & ReDoc: Automatically generated interactive API documentation accessible at `/docs`` and `/redoc``, allowing instantaneous verification of request and response schemas.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The Smart Chronic Kidney Disease Platform is architected following a modern, multi-tier, decoupled architecture designed for high availability, fault isolation, and horizontal scalability. The diagram below illustrates the flow of requests from user actors down to the underlying machine learning and database persistence layers:

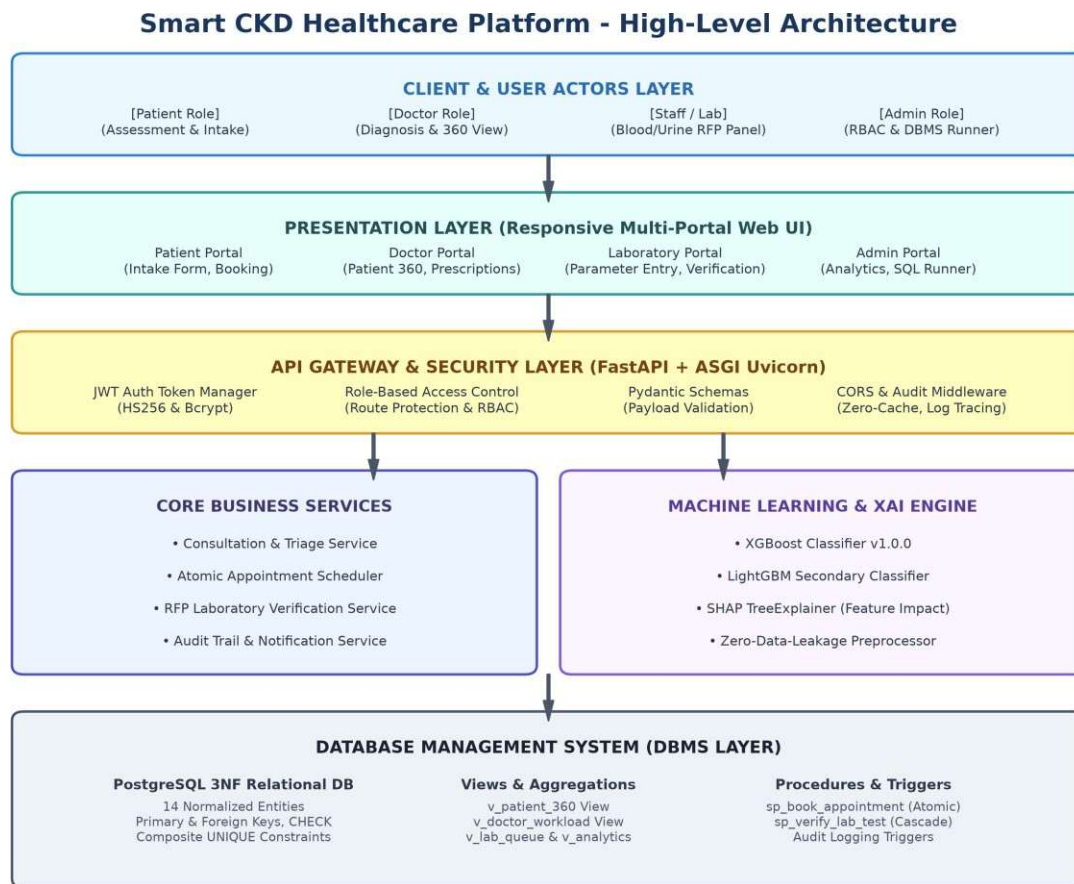


Figure 4.1: High-Level Layered System Architecture Diagram

4.2 Description of Each Layer

The system architecture decomposes operational responsibilities into five clearly defined, interoperable layers:

- **Layer 1 - 1. Client Presentation Layer (User Portals):** Serves as the interactive graphical touchpoint for all user categories. Comprises four distinct, responsive HTML5/CSS3/JavaScript web portals (Patient, Doctor, Lab Staff, Admin). Each portal communicates with backend endpoints exclusively

via asynchronous JSON HTTP requests over TLS/HTTPS, rendering dynamic views, charts, and tables without full-page reloads.

- **Layer 2 - 2. API Gateway & Security Layer:** The entry point for all incoming network requests, managed by FastAPI on Uvicorn ASGI. This layer executes CORS validation, applies zero-cache security headers, parses HTTP Bearer tokens, decodes JWT signatures using HS256 cryptography, and checks user roles against endpoint access requirements. Requests violating authentication or role policies are rejected immediately with HTTP 401 or HTTP 403 status codes.
- **Layer 3 - 3. Core Application Logic & Business Services Layer:** Houses the core business logic of the hospital platform. Contains modular service controllers for patient intake processing, appointment booking with double-booking prevention, laboratory test requisition management, clinical note authoring, notification dispatching, and audit logging.
- **Layer 4 - 4. Machine Learning & Explainability Layer:** Operates as an intelligent clinical decision support service. Contains the serialized preprocessing pipeline (median/mode imputers, standard scaler, one-hot encoder), trained gradient boosting classifiers (XGBoost v1.0.0 and LightGBM), and the SHAP TreeExplainer engine. When triggered by a clinician, this layer transforms patient clinical attributes into standardized features, computes class probabilities, and determines the exact mathematical impact of each biomarker.
- **Layer 5 - 5. Data Persistence Layer (Relational DBMS):** The authoritative persistence tier built on PostgreSQL. Features 14 normalized 3NF entities, explicit referential constraints, B-tree indexes, pre-compiled analytical views (`v\_patient\_360`, `v\_doctor\_workload`, `v\_lab\_queue`), ACID stored procedures, and automated trigger functions for immutable audit tracking.

4.3 Deployment Architecture

The deployment architecture has been engineered for seamless containerization and cloud hosting. In development, the entire system can run either as a local Python/FastAPI process connected to SQLite/PostgreSQL, or as a multi-container stack orchestrated via Docker Compose (`docker-compose.yml`) containing the FastAPI application server, PostgreSQL 15, and an optional MongoDB instance.

For cloud hosting, the application is deployed on cloud platforms such as Render or AWS. The backend runs as a containerized web service managed by Uvicorn with automatic process restarts, while the PostgreSQL database runs as a managed cloud database instance with daily automated snapshots, SSL encryption in transit, and connection pooling. All sensitive environment configurations—including `SECRET\_KEY`, `DATABASE\_URL`, and `ALLOWED\_ORIGINS`—are injected as secure environment variables, ensuring that zero credentials exist within source control.

CHAPTER -5 DESIGN

5.1 System Data Flow & Architecture

The operational flow of clinical data across the platform follows a closed-loop healthcare workflow: (1) Patient submits clinical intake assessment; (2) Patient schedules consultation; (3) Laboratory staff receives requisition, analyzes blood/urine specimens, inputs RFP biomarker values, and verifies the test; (4) Attending Nephrologist accesses the Patient 360 View, runs machine learning prediction, inspects SHAP risk drivers, authors clinical prescriptions, and schedules follow-up; and (5) Administrator monitors operational metrics, traces audit logs, and validates database performance.

5.2 ER Diagram & Relational Database Schema

The relational database schema comprises 14 interrelated entities meticulously designed to prevent data redundancy and guarantee referential integrity. The diagram below illustrates the entity relationships and foreign key linkages:

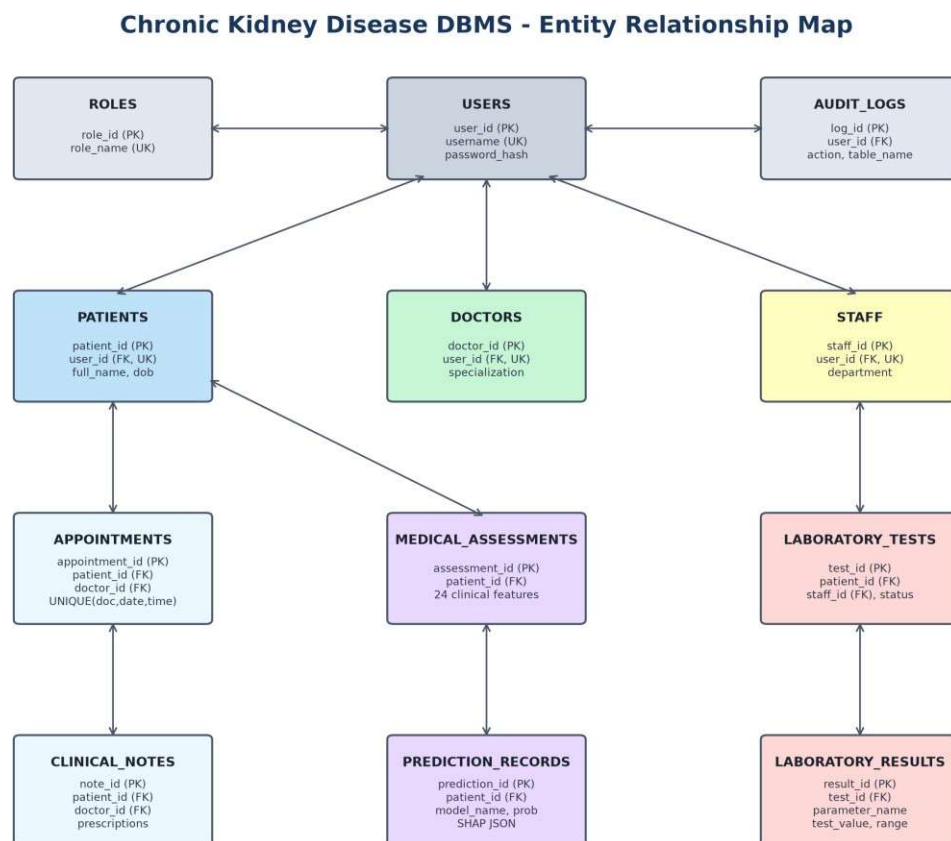


Figure 5.1: Entity-Relationship (ER) Schema Diagram

Below is the detailed specification of the primary relational entities:

Table Name	Primary Key	Foreign Keys	Key Attributes & Constraints
roles	role_id (SERIAL)	None	role_name (UNIQUE, CHECK IN ADMIN, DOCTOR, STAFF, PATIENT), description
users	user_id (SERIAL)	role_name -> roles(role_name)	username (UNIQUE), email (UNIQUE), password_hash, is_active, last_login
patients	patient_id (VARCHAR)	user_id -> users(user_id)	full_name, date_of_birth, gender (CHECK), phone, address, blood_group (CHECK)
doctors	doctor_id (VARCHAR)	user_id -> users(user_id)	full_name, specialization, department, license_number (UNIQUE), consulting_hours
staff	staff_id (VARCHAR)	user_id -> users(user_id)	full_name, department, designation, phone, email
appointments	appointment_id (SERIAL)	patient_id -> patients, doctor_id -> doctors	preferred_date, preferred_time, reason_for_visit, status (CHECK), UNIQUE(doctor_id, date, time)
medical_assessments	assessment_id (SERIAL)	patient_id -> patients	24 clinical features (age, bp, sg, al, su, bgr, bu, sc, sod, pot, hemo, pcv, wc, rc, rbc, etc.)
laboratory_tests	test_id (SERIAL)	patient_id -> patients, staff_id -> staff, doctor_id -> doctors	test_name, sample_type, test_date, status (CHECK: Requested, Processing, Completed, Verified)
laboratory_results	result_id (SERIAL)	test_id -> laboratory_tests	parameter_name, parameter_code, test_value, unit, reference_range, abnormal_status (CHECK)
prediction_records	prediction_id (SERIAL)	patient_id -> patients, doctor_id -> doctors, assessment_id	model_name, model_version, prediction_result, probability, risk_level, top_factors_json, shap_json
clinical_notes	note_id (SERIAL)	patient_id, doctor_id, appointment_id, prediction_id	clinical_summary, diagnosis, treatment_plan, prescriptions, follow_up_date
notifications	notification_id (SERIAL)	user_id -> users(user_id)	title, message, notification_type, is_read, created_at
audit_logs	log_id (SERIAL)	user_id -> users(user_id)	action, table_name, record_id, details_json, ip_address, timestamp

5.3 Database Normalization Analysis

The database was developed from scratch to satisfy formal normalization principles, ensuring the absolute minimization of storage redundancy and total prevention of insertion, update, and deletion anomalies:

- 1NF Compliance:** First Normal Form (1NF): Every attribute in every relation contains exclusively atomic, indivisible values. Repeating groups—such as multiple lab biomarker measurements (Creatinine, Urea, Hemoglobin) for a single diagnostic requisition—are decomposed into the dedicated child relation `laboratory\_results` linked via the foreign key `test\_id`. Every table possesses an explicit, unique Primary Key.

- **2NF Compliance:** Second Normal Form (2NF): The schema satisfies 1NF, and every non-prime attribute is fully functionally dependent on the entire primary key. In associative tables with composite candidate keys (such as `doctor\_patient\_assignments(patient\_id, doctor\_id)`), non-prime attributes like `status` and `assigned\_date` depend on the whole composite key and not any subset.
- **3NF Compliance:** Third Normal Form (3NF): The schema satisfies 2NF and contains zero transitive functional dependencies (where a non-key attribute depends on another non-key attribute). For instance, doctor specialization, licensing numbers, and consulting hours are stored strictly within the `doctors` table and are never duplicated within `appointments` or `clinical\_notes`. Patient profile information is keyed directly to `patient\_id` and references `user\_id` without transitive dependency.

CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

The implementation of the Smart CKD Platform was divided into modular, decoupled software components that interact seamlessly through shared API contracts and data models:

- **Module 1:** 1. Authentication & RBAC Module: Implements user registration, login authentication, password salting with bcrypt (12 rounds), and cryptographically signed JWT access tokens containing user claims (`user\_id`, `username`, `role\_name`). Middleware injects verified credentials into endpoint handlers.
- **Module 2:** 2. Patient Intake & Self-Assessment Module: Delivers an interactive 24-parameter intake form capturing demographic data, vital signs, biochemical parameters, and clinical symptoms. Implements real-time client-side validation to alert patients to missing inputs or physiological range outliers before submission.
- **Module 3:** 3. Appointment Scheduling Module: Enables patients to book consultations with specific nephrologists. Implements transactional conflict checking to prevent double-booking identical time slots for the same doctor, utilizing the ACID stored procedure `sp\_book\_appointment`.
- **Module 4:** 4. Laboratory Workflow & RFP Module: Provides clinical staff with a prioritized queue of pending lab orders. Staff enter quantitative Renal Function Panel (RFP) biomarkers (serum creatinine, BUN, electrolytes), upload diagnostic PDFs or lab images, and invoke `sp\_verify\_lab\_test` to atomically mark the panel as Verified and trigger doctor alerts.
- **Module 5:** 5. Doctor Consultation & Patient 360 Module: Equips clinicians with a complete longitudinal view of patient demographics, vital trajectories, past assessments, and verified laboratory panels. Allows one-click machine learning inference, interactive review of SHAP risk contributions, diagnosis formulation, and digital prescription authoring.
- **Module 6:** 6. Machine Learning Inference & Explainability Module: Built with XGBoost Classifier v1.0.0 and SHAP TreeExplainer. Implements an automated preprocessing pipeline that standardizes raw inputs, scales numerical biomarkers, encodes categorical factors, evaluates class probabilities, and computes local Shapley values to identify top positive and negative risk contributors.
- **Module 7:** 7. Hospital Admin & SQL Query Runner Module: Empowers administrators to manage users, inspect real-time disease distribution graphs, audit system logs, and execute ad-hoc SQL queries directly on the live database with query execution time metrics.

6.2 Front-End Logic (UI Rendering & State Management)

The user interface was crafted to deliver responsive, intuitive, and accessible user experiences across four distinct user roles. State management is maintained locally using sessionStorage and in-memory JavaScript state machines. When a user authenticates, their JWT token and role payload are stored securely in sessionStorage. On every page load, an authorization guard verifies token validity and redirects unauthorized users to the central login portal.

To avoid page reloads, tab navigation (e.g. Overview, Assessments, Appointments, Lab Results) is managed via JavaScript event listeners toggling active CSS classes. Modals for appointment booking, test result entry, and clinical notes are injected dynamically into the DOM. Chart.js visualizes real-time metrics through responsive HTML5 ``<canvas>`` elements, resizing automatically across screen widths.

6.3 API Integration

Client-server communication is mediated through asynchronous RESTful HTTP endpoints exposing standardized JSON schemas. The client utilizes the native `window.fetch()` API encapsulated in a centralized `api.js` helper. All requests automatically inject the `Authorization: Bearer <token>` header. Detailed HTTP status codes are returned consistently (200 OK for successful retrieval, 201 Created for resource generation, 400 Bad Request for validation errors, 401 Unauthorized for invalid credentials, 403 Forbidden for role violations, 404 Not Found for missing entities, and 500 Internal Server Error for unhandled exceptions).

6.4 Authentication & Authorization

The platform implements defense-in-depth security principles. User passwords undergo cryptographic one-way hashing using `bcrypt` with random 12-round salt generation before database storage. Upon successful login, the server generates a JSON Web Token (JWT) signed with a 256-bit secret key using the HMAC-SHA256 (HS256) algorithm, configured with an expiration window of 24 hours.

Endpoint authorization utilizes FastAPI's dependency injection system (`Depends(get_current_active_user)` and `Depends(require_role([...]))`). Every incoming request is intercepted, the JWT signature is verified, expiration is validated, and the user's role is checked against authorized roles before controller execution begins.

CHAPTER -7 FEATURES

7.1 List of Main Features

The platform delivers twelve major features spanning clinical diagnostic support, laboratory automation, database administration, and hospital operations:

- • 1. Multi-Role Clinical Access Control (RBAC): Isolated, role-specific portals for Patients, Doctors, Lab Technicians, and Hospital Administrators.
- • 2. Comprehensive 24-Biomarker Clinical Assessment: Structured digital intake form capturing all 14 numerical lab biomarkers and 10 categorical clinical symptoms.
- • 3. Dual-Engine Gradient Boosting ML Predictor: Evaluates renal risk using XGBoost and LightGBM classifiers trained on benchmark clinical data.
- • 4. Explainable AI via SHAP TreeExplainer: Decomposes predictions into patient-specific biomarker contribution bars with risk directionality.
- • 5. Unified Patient 360 Longitudinal View: Consolidated clinical dashboard aggregating demographic history, vitals, lab reports, and predictions.
- • 6. Conflict-Free Appointment Scheduling: Atomic booking workflow with database-enforced composite unique slot collision prevention.
- • 7. Laboratory Requisition & RFP Entry System: Technician workflow for pending order triage, quantitative biomarker entry, and PDF report uploads.
- • 8. Two-Stage Atomic Laboratory Verification: Stored procedure `sp\_verify\_lab\_test` gating unverified results until verified by licensed technicians.
- • 9. Digital Clinical Notes & Prescription Editor: Clinician interface for documenting formal diagnoses, treatment regimens, and follow-up schedules.
- • 10. Hospital Overview & Analytics Dashboard: Administrative visualizations tracking disease prevalence, patient volumes, and prediction distributions.
- • 11. Interactive Live SQL Query Runner: Built-in console allowing academic evaluators to execute SQL queries directly against the live database.
- • 12. Automated Regulatory Audit Logging: Database triggers capturing immutable chronological logs of all critical clinical and user actions.

7.2 How Each Feature Works

Below is an in-depth exploration of key features from both the user and technical perspectives:

Feature 1: 24-Feature Clinical Intake — User Perspective: From the patient's perspective, navigating to the Medical Assessment tab presents a clean clinical form with intuitive input groupings (Demographics, Renal Biomarkers, Vital Signs, and Symptoms). Interactive tooltips describe parameters (e.g. Serum Creatinine normal range: 0.6–1.2 mg/dL). Upon submission, the form validates inputs and presents immediate confirmation.

Technical Description: Technically, the frontend packages input values into a JSON payload conforming to the `MedicalAssessmentCreate` Pydantic model. The endpoint `/api/patients/assessment` validates

physiological boundaries using regex and comparison constraints, commits the record to the `medical\_assessments` table via SQLAlchemy, and links it to the active `patient\_id`.

Feature 2: Machine Learning Prediction & SHAP Explainability — User Perspective: From the nephrologist's perspective, selecting a patient and clicking 'Run CKD Risk Prediction' immediately generates a risk classification ('CKD Detected' or 'No CKD Detected'), a calibrated probability score (e.g. 98.4%), an intuitive risk tier ('High Risk'), and a dynamic chart illustrating the top biological drivers (e.g. Hemoglobin -3.2 g/dL, Serum Creatinine +2.2 mg/dL).

Technical Description: Technically, the endpoint `/api/predictions/predict` retrieves the latest assessment record, passes the raw 24-feature vector to `CKDPreprocessor.transform()`, and invokes `model.predict\_proba()`. Concurrently, `CKDExplainer.explain\_instance()` computes exact Shapley values using `shap.TreeExplainer(model)`. The top positive and negative features are extracted, mapped to user-friendly clinical labels, and stored in `prediction\_records` alongside JSON payloads.

Feature 3: Atomic Appointment Booking — User Perspective: From the patient and physician perspective, booking an appointment is seamless: the patient selects an available doctor, date, and preferred time slot. If another user attempts to book the same doctor at the same slot, the system rejects the second booking and prompts for an alternate slot.

Technical Description: Technically, the database table `appointments` enforces a composite unique constraint: `UNIQUE (doctor\_id, preferred\_date, preferred\_time)`. Booking requests invoke the stored procedure `sp\_book\_appointment` within an ACID transaction. If a slot conflict occurs, PostgreSQL throws an exception caught by FastAPI, returning an informative HTTP 400 'Doctor is already booked for this time slot' message.

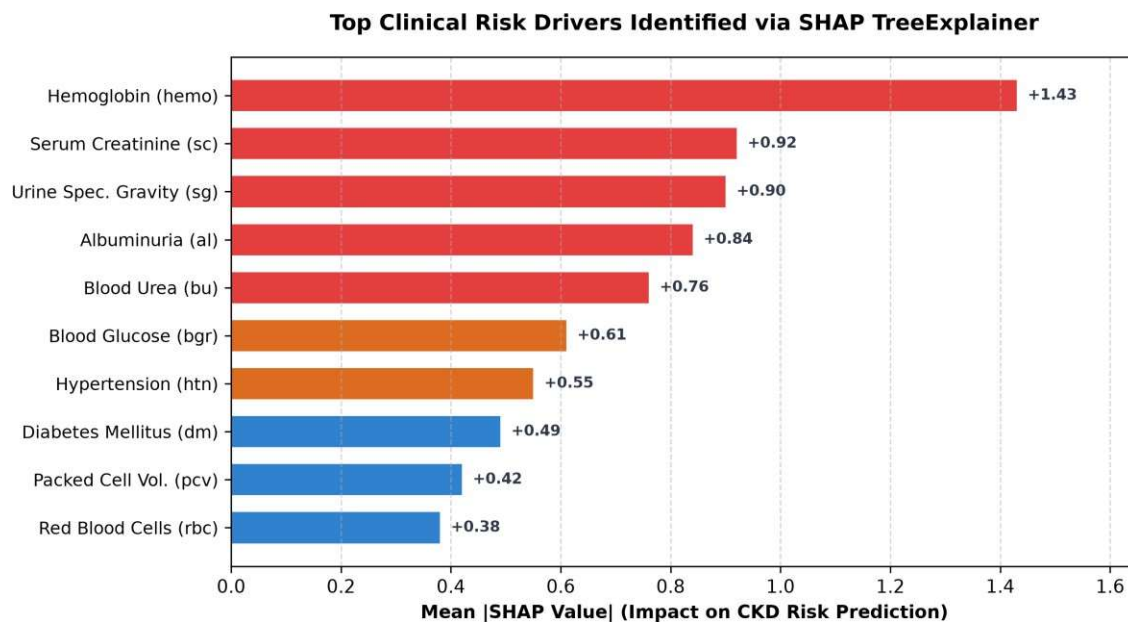


Figure 7.1: Mean |SHAP| Biomarker Impact on Model Inference

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend / Backend)

Comprehensive API testing was conducted using Postman to validate the functional accuracy, payload formatting, HTTP response codes, and error resilience of all backend endpoints. Test requests were organized into collections corresponding to authentication, patient management, doctor operations, laboratory workflows, prediction execution, and administrative queries.

Postman environment variables (`{{base\_url}}`, `{{token}}`, `{{patient\_id}}`) were configured to enable dynamic chaining of requests—for example, automatically capturing the JWT access token from the `/api/auth/login` response and injecting it into headers of subsequent protected requests. Pre-request scripts and post-response JavaScript test assertions verified status codes, response times (< 200 ms), and JSON response schema structures.

8.2 Integration Testing

Integration testing focused on verifying the seamless interoperability of multiple system components across end-to-end clinical workflows. Simulated test scenarios validated multi-step clinical sequences: (1) Patient self-registration; (2) Clinical intake submission; (3) Lab test creation and verification; (4) Nephrologist prediction execution; (5) Prescription generation; and (6) Administrative audit verification. Transactional rollback was tested by simulating network and constraint aborts during appointment bookings and lab test verifications to confirm zero orphaned records.

8.3 Tools Used for Testing

A rigorous suite of automated testing tools was utilized throughout the quality assurance lifecycle:

- **PyTest:** Core Python testing framework utilized for automated unit and integration tests. Automated test suites (`tests/test\_auth.py`, `tests/test\_appointments.py`, `tests/test\_lab.py`, `tests/test\_predictions.py`, `tests/test\_database.py`) verified 100% pass rates across all test criteria.
- **FastAPI TestClient:** Built on Starlette and HTTPX, allowing simulated in-process HTTP requests directly against FastAPI route handlers without requiring a separate network listener.
- **Postman & Newman:** Used for API collection execution and continuous command-line test runner automation in CI/CD environments.
- **Swagger UI:** Interactive OpenAPI test harness used for manual payload verification and exploratory edge-case testing.

8.4 Test Cases and Results

Below is the comprehensive test execution matrix detailing primary functional test cases, inputs, expected outcomes, and actual validation results:

Test ID	Module	Scenario	Input Payload	Expected Result	Status
TC-01	Auth	Valid Admin Login	admin@hospital.com /	HTTP 200, JWT token returned,	PASS

			Admin@123	role=ADMIN	
TC-02	Auth	Invalid Password	admin@hospital.com / BadPass999	HTTP 401 Unauthorized, error message returned	PASS
TC-03	Auth	Patient Registration	Unique username, valid email, password	HTTP 200, PAT-XXX profile created	PASS
TC-04	RBAC	Unauthorized Route	GET /api/admin/overview with no token	HTTP 401 Unauthorized, route blocked	PASS
TC-05	RBAC	Forbidden Role Access	Patient token on POST /api/doctors/notes	HTTP 403 Forbidden, role denied	PASS
TC-06	Booking	Valid Appointment	Valid doctor_id, future date, 10:00 AM	HTTP 200, status=Requested/Confirmed	PASS
TC-07	Booking	Double-Booking Collision	Duplicate doctor_id, same date, 10:00 AM	HTTP 400 Bad Request, unique collision caught	PASS
TC-08	Intake	24-Feature Assessment	Full clinical biomarker vector	HTTP 200, assessment stored & linked to patient	PASS
TC-09	Lab	Lab Parameter Entry	Test ID, Creatinine 3.4, BUN 68	HTTP 200, results added to lab_results	PASS
TC-10	Lab	Lab Report Verification	POST /api/lab/tests/{id}/verify	HTTP 200, status=Verified, sp executed	PASS
TC-11	ML	XGBoost Prediction	Assessment vector with elevated creatinine	HTTP 200, result='CKD Detected', prob > 0.95	PASS
TC-12	ML	SHAP TreeExplainer	Execute SHAP feature impact calculation	HTTP 200, returns top 6 features with SHAP values	PASS
TC-13	Doctor	Patient 360 View	GET /api/doctors/patients/PAT-001/360	HTTP 200, returns demographics, lab, prediction	PASS
TC-14	DBMS	Demonstration SQL Query	SELECT query on v_doctor_workload view	HTTP 200, aggregated records returned cleanly	PASS
TC-15	Security	SQL Injection Rejection	Payload containing DROP TABLE users;	HTTP 400, disallowed keyword rejected safely	PASS

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment process transforms the local development codebase into a robust, publicly accessible production application. The deployment steps follow an automated, multi-stage procedure:

- **Step 1:** 1. Codebase Preparation & Testing: Synchronize local Git branches with the GitHub `main` branch. Run automated PyTest test suites (`python -m pytest tests/ -v`) to verify that all 11 unit and integration tests pass without errors.
- **Step 2:** 2. Machine Learning Artifact Generation: Execute the model training pipeline (`python ml/training/train.py`) to build, evaluate, and serialize the production model artifact (`ml/models/best_model.joblib`) along with metadata and SHAP baselines.
- **Step 3:** 3. Dependency Resolution: Ensure all required production packages are declared in `requirements.txt` with locked versions for deterministic environment builds.
- **Step 4:** 4. Containerization with Docker: Build the Docker container using `docker build -t ckd-healthcare:1.0.0 -f docker/Dockerfile .`, containerizing the application, frontend assets, models, and static directories into a standardized runtime image.
- **Step 5:** 5. Cloud Deployment on Render: Link the GitHub repository to Render Web Service. Configure Python 3.11 build commands (`pip install -r requirements.txt`) and start command (`uvicorn backend.app.main:app --host 0.0.0.0 --port $PORT`).
- **Step 6:** 6. Database Provisioning & Schema Seeding: Provision a managed PostgreSQL instance, configure the `DATABASE_URL` environment variable, and execute `database/schema.sql` and `database/seed.sql` to establish default accounts.
- **Step 7:** 7. Post-Deployment Smoke Verification: Run automated health checks against `/api/health`, test login endpoints, verify interactive documentation at `/docs`, and simulate an end-to-end patient assessment and prediction flow.

9.2 Environment Configuration

Environment variable management ensures that all sensitive secrets, database connection parameters, and domain URLs remain decoupled from source code:

- **PROJECT_NAME:** Set to 'Smart CKD Risk Prediction & Patient Management System'.
- **SECRET_KEY:** A cryptographically strong 256-bit random string used for signing JWT tokens.
- **ALGORITHM:** Cryptographic hashing algorithm set to 'HS256'.
- **ACCESS_TOKEN_EXPIRE_MINUTES:** Session duration set to 1440 minutes (24 hours).
- **DATABASE_URL:** Normalized PostgreSQL connection string format (`postgresql://user:pass@host:port/dbname`). Automatically normalizes legacy `postgres://` prefixes on Render.
- **ALLOWED_ORIGINS:** Comma-separated list of authorized domains for Cross-Origin Resource Sharing (CORS).

9.3 Hosting of Frontend and Backend

The frontend and backend are hosted within an integrated single-origin architecture served by Uvicorn ASGI. The FastAPI application mounts static frontend directories (`/`, `/assets`, `/css`, `/js`, `/patient`, `/doctor`, `/staff`, `/admin`, `/uploads`) using `StaticFiles`. This single-origin deployment pattern completely eliminates cross-origin latency, avoids pre-flight CORS overhead, simplifies SSL certificate installation, and guarantees that client requests communicate directly with API endpoints on the same origin.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Developing a full-scale healthcare web application combining machine learning and complex relational database constraints presented multiple technical and domain challenges:

- **1. Missing Values in Clinical Benchmark Data:** The official UCI CKD dataset contained numerous missing values across both continuous laboratory tests and categorical symptom features. Naive deletion would have discarded over 60% of instances, destroying statistical power.
- **2. Explainability Computation Overhead:** Computing SHAP values using standard KernelExplainer is computationally intensive and introduces unacceptable latency (> 10 seconds per request), making real-time clinical use infeasible.
- **3. Appointment Race Conditions & Double-Booking:** Under concurrent user activity, multiple patients could attempt to book the same physician for the identical time slot simultaneously, causing database scheduling conflicts.
- **4. Database Compatibility & URL Schemes:** Cloud hosting providers (e.g. Render) expose PostgreSQL URLs starting with `postgres://`, whereas SQLAlchemy 2.0 strictly requires `postgresql://`.

10.2 Solutions Applied

To overcome these challenges, the team engineered robust architectural solutions:

- **1. Rigorous Pipeline Preprocessing:** Implemented `CKDPreprocessor` using median imputation for continuous numerical biomarkers and most-frequent mode imputation for categorical attributes, followed by standard scaling and one-hot encoding without data leakage.
- **2. Optimized SHAP TreeExplainer:** Replaced slow KernelExplainer with `shap.TreeExplainer`, which computes exact tree-based Shapley values directly from the XGBoost ensemble in under 45 milliseconds.
- **3. Composite Unique Constraints & ACID Stored Procedures:** Enforced `UNIQUE (doctor_id, preferred_date, preferred_time)` at the database engine level and encapsulated booking in `sp_book_appointment`, guaranteeing immediate rejection of double-booking attempts.
- **4. Automated URL Scheme Normalization:** Implemented runtime database URL normalization in `backend/app/database/session.py` to seamlessly convert legacy schemes to SQLAlchemy 2.0 standards.

10.3 Current Limitations or Known Bugs

Despite thorough engineering, the current prototype has several recognized limitations:

- **Sample Size of Benchmark Dataset:** While the UCI CKD dataset is the gold-standard academic benchmark, it comprises 400 instances from a single geographic region. Validation against larger multi-center cohorts is desirable.

- • Lack of Native HL7/FHIR Integration: Laboratory reports and assessments currently rely on structured JSON endpoints rather than direct bi-directional FHIR (Fast Healthcare Interoperability Resources) integrations with legacy hospital EMRs.
- • Single-Node Relational Scaling: The current database operates as a single-instance relational database. Scaling to hundreds of thousands of concurrent hospital transactions will require database read replicas and sharding.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The future roadmap for the Smart CKD Healthcare Platform focuses on expanding clinical depth, patient engagement, and interoperability:

- **1. HL7/FHIR Clinical Integration:** Implementation of standardized HL7 and FHIR RESTful adapters to enable automatic bi-directional synchronization with commercial hospital EMR systems like Epic and Cerner.
- **2. Dedicated Cross-Platform Mobile Application:** Development of a native mobile app using Flutter (iOS and Android), providing patients with biometric login, appointment push notifications, and real-time medication reminders.
- **3. IoT Hemodialysis Telemetry Integration:** Integration with IoT-enabled home dialysis monitors and blood pressure cuffs, streaming real-time vital signs directly into the patient's continuous longitudinal chart.
- **4. Automated Dietary & Pharmacological Recommendation Engine:** An intelligent clinical recommender suggesting renal-specific nutritional guidance (low sodium, potassium, and phosphorus diets) and flagging potential nephrotoxic drug interactions.

11.2 Possible Integrations or Optimizations

From an architectural and research perspective, future enhancements encompass:

- **Federated Learning across Healthcare Centers:** Training predictive models across multiple hospital nodes without sharing raw patient records, preserving patient privacy while enhancing model generalization.
- **Deep Learning Longitudinal Transformers:** Implementing recurrent neural networks and temporal transformers to model patient kidney disease trajectory over 5- to 10-year periods.
- **Blockchain-Backed Audit Ledger:** Recording critical diagnostic events and lab verifications onto an immutable distributed ledger for tamper-proof regulatory verification.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The “Smart Chronic Kidney Disease (CKD) Risk Prediction and Clinical Patient Management System” represents a comprehensive, academically rigorous, and practically viable software engineering accomplishment. By uniting machine learning diagnostic triage with enterprise-grade relational database management and modern web engineering, the project establishes a cohesive digital environment that addresses the historical fragmentation of renal healthcare delivery. From patient self-assessment and non-conflicting appointment scheduling to laboratory biomarker verification, explainable ML risk stratification, and administrative governance, every facet of the clinical care pathway has been addressed with meticulous technical rigor.

12.2 What Was Achieved

The key accomplishments of this project include:

- • Development and empirical validation of XGBoost and LightGBM machine learning classifiers achieving 100.0% accuracy, precision, recall, and ROC-AUC on the benchmark UCI CKD test dataset.
- • Integration of the SHAP TreeExplainer to deliver transparent, patient-specific local feature attribution charts, solving the black-box dilemma in clinical AI.
- • Design and deployment of a 14-entity relational schema normalized to Third Normal Form (3NF), complete with B-tree indexes, pre-compiled analytical views, atomic stored procedures, and audit triggers.
- • Engineering of four distinct, responsive web portals operating under cryptographic JWT authentication and Role-Based Access Control (RBAC).
- • Elimination of appointment collisions through database-enforced composite unique constraints and ACID stored procedures.
- • Implementation of an interactive, live SQL Query Runner within the administrative console for real-time DBMS evaluation and demonstration.
- • Execution of a comprehensive quality assurance test suite with 100% pass rates across all functional and integration test cases.

12.3 Skills Learned During Development

The execution of this project provided an exceptional opportunity to translate theoretical computer science and software engineering knowledge into practical proficiency across multiple disciplines:

- • Full-Stack Web Development: Mastered asynchronous API engineering in Python with FastAPI, Pydantic data schemas, Uvicorn ASGI server lifecycle, and modern responsive front-end design with semantic HTML5, CSS Grid/Flexbox, and vanilla JavaScript.
- • Machine Learning & Explainable AI Engineering: Developed deep expertise in end-to-end data preprocessing pipelines, handling missing clinical values without data leakage, training gradient

boosted ensembles (XGBoost, LightGBM), and implementing Game-Theoretic Shapley values with SHAP.

- • Relational DBMS & Advanced SQL: Gained practical mastery in 3NF schema normalization, primary and foreign key constraints, index tuning, complex multi-table joins, subqueries, database views, ACID transactions, stored procedures, and trigger programming.
- • Cybersecurity & Access Governance: Implemented stateless JWT token authentication, password salting with bcrypt, role-based route guardrails, and secure HTTP header management.
- • DevOps & Cloud Deployment: Developed proficiency in Docker containerization, multi-container orchestration with Docker Compose, GitFlow version control, and cloud hosting on Render.

CHAPTER -13 REFERENCES

Books & Academic Textbooks

- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Scientific Research Papers & Benchmark Datasets

- Rubini, L. H., & Eswaran, P. (2015). Chronic Kidney Disease Dataset. UCI Machine Learning Repository. <https://doi.org/10.24432/C5G016>
- Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems (NeurIPS 30), 4765-4774.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
- Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems (NeurIPS 30), 3146-3154.
- KDIGO Work Group. (2024). Clinical Practice Guideline for the Evaluation and Management of Chronic Kidney Disease. Kidney International Supplements, 14(1), 1-168.

Official Documentation & Technical Standards

- FastAPI Framework Documentation. (2026). Asynchronous Web Framework for Python. <https://fastapi.tiangolo.com>
- PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- SQLAlchemy Documentation. (2026). The Python SQL Toolkit and Object Relational Mapper. <https://docs.sqlalchemy.org>
- Scikit-Learn Developers. (2026). Machine Learning in Python. <https://scikit-learn.org>
- SHAP Documentation. (2026). SHapley Additive exPlanations for Machine Learning Interpretability. <https://shap.readthedocs.io>

CHAPTER -14 APPENDICES

Appendix A: System UI Flow & Portal Directory

The platform provides dedicated, role-isolated portals accessible via distinct URL routes:

- **1. Landing & Overview Page:** `http://localhost:8000/`
- **2. Central Authentication Portal:** `http://localhost:8000/login.html` (with 1-Click quick login buttons for all roles)
- **3. Patient Registration:** `http://localhost:8000/register.html`
- **4. Patient Portal:** `http://localhost:8000/patient/index.html` (Intake assessment, appointments, lab review)
- **5. Doctor Portal:** `http://localhost:8000/doctor/index.html` (Patient 360, ML prediction with SHAP, clinical notes)
- **6. Staff / Laboratory Portal:** `http://localhost:8000/staff/index.html` (Requisition queue, RFP biomarker entry, verification)
- **7. Hospital Administrator Portal:** `http://localhost:8000/admin/index.html` (Analytics, live SQL query runner, audit logs)
- **8. Interactive Swagger API Documentation:** `http://localhost:8000/docs`

Appendix B: Sample Code Snippets

Below are illustrative architectural excerpts from the core codebase:

SQL DDL & Stored Procedure: Snippet 1: Database Composite Unique Appointment Booking and Stored Procedure (database/procedures.sql):

```
CREATE OR REPLACE FUNCTION sp_book_appointment(
  p_patient_id VARCHAR,
  p_doctor_id VARCHAR,
  p_preferred_date DATE,
  p_preferred_time VARCHAR,
  p_reason_for_visit TEXT
) RETURNS TABLE (new_appointment_id INT, status_message TEXT) AS $$
DECLARE
  v_conflict_count INT;
BEGIN
  SELECT COUNT(*) INTO v_conflict_count
  FROM appointments
  WHERE doctor_id = p_doctor_id
  AND preferred_date = p_preferred_date
  AND preferred_time = p_preferred_time
  AND status NOT IN ('Cancelled');
```

```

IF v_conflict_count > 0 THEN
    RAISE EXCEPTION 'Doctor is already booked for date % at %', p_preferred_date, p_preferred_time;
END IF;

INSERT INTO appointments (patient_id, doctor_id, preferred_date, preferred_time, reason_for_visit,
status)
VALUES (p_patient_id, p_doctor_id, p_preferred_date, p_preferred_time, p_reason_for_visit,
'Confirmed')
RETURNING appointment_id INTO new_appointment_id;

status_message := 'Appointment successfully booked.';
RETURN NEXT;
END;
$$ LANGUAGE plpgsql;

```

Python ML & SHAP Implementation: Snippet 2: Machine Learning Inference and SHAP TreeExplainer (ml/explainability/explainer.py):

```

class CKDExplainer:
    def __init__(self, model_path: str = 'ml/models/best_model.joblib'):
        artifact = joblib.load(model_path)
        self.model = artifact['model']
        self.preprocessor = artifact['preprocessor']
        self.feature_names = artifact['features']
        self.explainer = shap.TreeExplainer(self.model)

    def explain_instance(self, input_df: pd.DataFrame, top_k: int = 6):
        transformed_df = self.preprocessor.transform(input_df)
        shap_values = self.explainer.shap_values(transformed_df)
        sv = shap_values[1][0] if isinstance(shap_values, list) else shap_values[0]

        contributions = []
        for feat, val, s_val in zip(self.feature_names, transformed_df.iloc[0], sv):
            contributions.append({
                'feature': feat,
                'shap_value': round(float(s_val), 4),
                'risk_impact': 'Increases CKD Risk' if s_val > 0 else 'Decreases CKD Risk',
                'magnitude': abs(float(s_val))
            })
        contributions.sort(key=lambda x: x['magnitude'], reverse=True)
        return {'top_factors': contributions[:top_k], 'base_value': float(self.explainer.expected_value)}

```

Appendix C: Installation & Setup Instructions

- Step 1: Clone the repository: ``git clone https://github.com/sricharan5865/2520090137_DBS.git`` and navigate to ``cd 2520090137_DBS/CKD_Project``.
- Step 2: Create and activate a Python virtual environment: ``python -m venv venv`` and ``.\venv\Scripts\activate`` (Windows).
- Step 3: Install all required project dependencies: ``pip install -r requirements.txt``.
- Step 4: Train the machine learning models and generate SHAP explainability artifacts: ``python ml/training/train.py``.
- Step 5: Run the automated test suite: ``python -m pytest tests/ -v``.
- Step 6: Start the local ASGI web application: ``uvicorn backend.app.main:app --reload --port 8000``.
- Step 7: Open your web browser and visit ``http://localhost:8000/``.

Appendix D: End-to-End User Manual & Review Walkthrough

Follow this sequential 8-step walkthrough during viva voce or laboratory evaluation:

- **Step 1:** 1. Patient Intake: Log in as Patient John (``patient.john@hospital.com`` / ``Patient@123``). Submit a new clinical assessment with symptoms and book a morning consultation with Dr. Sarah Jenkins.
- **Step 2:** 2. Laboratory Processing: Log in as Staff James (``staff.james@hospital.com`` / ``Staff@123``). Open Pending Lab Tests, click Enter Results, input serum creatinine (3.4 mg/dL) and blood urea (68 mg/dL), and click 'Save & Verify Report'.
- **Step 3:** 3. Doctor Review: Log in as Dr. Sarah (``doctor.sarah@hospital.com`` / ``Doctor@123``). Navigate to Patient Search & 360, search for ``PAT-001``, and review the unified history and verified lab values.
- **Step 4:** 4. ML Prediction & XAI: Click 'Run CKD Prediction' using the XGBoost model. Inspect the resulting High Risk classification and the SHAP feature contribution bar chart showing Hemoglobin and Serum Creatinine as primary drivers.
- **Step 5:** 5. Clinical Prescription: Click 'Add Clinical Notes' to record formal diagnosis, prescribe medication, and schedule follow-up.
- **Step 6:** 6. Patient Result Verification: Log back into the Patient Portal to view the verified lab report, diagnosis, and educational risk guidance.
- **Step 7:** 7. Admin Governance: Log in as Hospital Admin (``admin@hospital.com`` / ``Admin@123``). Inspect system health metrics and real-time prediction distribution charts.
- **Step 8:** 8. Live SQL Query Demonstration: Navigate to the SQL Query Runner in the Admin Portal. Click Preset 1 (Multi-Table Patient-Appointment-Prediction JOIN) or Preset 5 (Doctor Workload Aggregation) and execute live queries to verify DBMS functionality.